

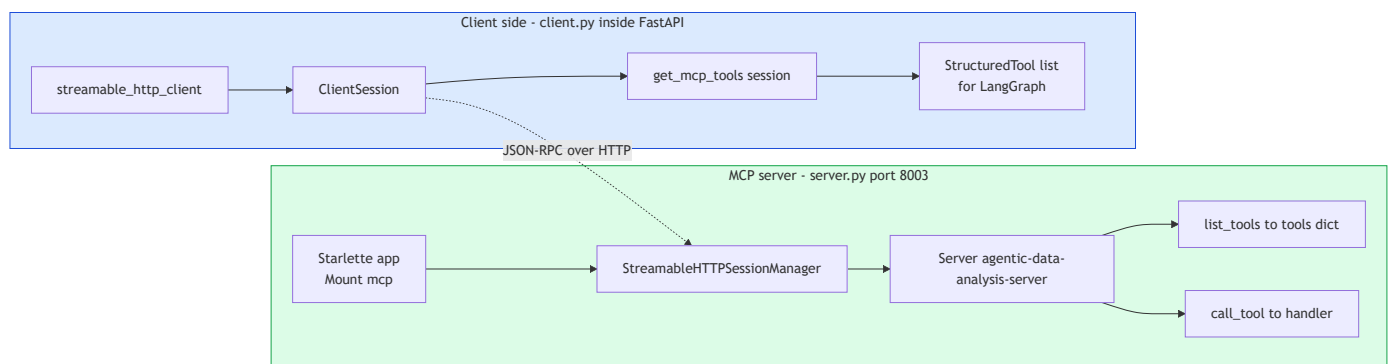
- 03 — MCP Server and Client
 - What is MCP, in one paragraph?
 - The two halves
 - The server: server.py
 - Running it
 - The client bridge: client.py
 - JSON Schema → Pydantic
 - None-filtering
 - Transport: streamable HTTP
 - Why this split?

03 — MCP Server and Client

What is MCP, in one paragraph?

Model Context Protocol is just a contract for "this server exposes a set of named functions; this client can list them and call them." Think of it as RPC dressed up so any LLM client can discover and invoke your tools without extra glue. We use the **streamable-HTTP** transport: one long-lived HTTP connection carrying JSON-RPC frames in both directions.

The two halves



The server: `server.py`

A minimal Starlette app that wraps a low-level `mcp.server.lowlevel.Server` behind a streamable-HTTP transport.

```
server.py

Server("agentic-data-analysis-server")
    @list_tools → tools/__init__.tools
    @call_tool  → tools[name]["handler"]

StreamableHTTPSessionManager(server)
    └─ Mount("/mcp", ...)

uvicorn.run(app, host=0.0.0.0, port=8003)
```

Key points:

- **Two handlers only** — `list_tools` returns descriptors, `call_tool` dispatches by name. All domain logic lives in `tools/*.py`.
- **Tool registry** (`tools/__init__.py`) is a plain dict mapping `tool_name` → `{tool: types.Tool, handler: callable}`. Adding a new tool means importing it and adding one entry — `server.py` and `client.py` need no edits.
- **Lifespan** prints the registered tool names at startup; there is no other initialisation beyond that.

Running it

```
python python/agentic/server.py
# 🏗️ Agentic Data Analysis MCP Server starting...
#   Tools registered: ['get_db_schema', 'query_mill_data', ...]
# 🚀 MCP Server is running on port 8003 (/mcp)
```

The server is idempotent — every DataFrame it loads stays in memory for the lifetime of the process, so subsequent tool calls during the same analysis are fast and free of database traffic.

The client bridge: `client.py`

`client.py` contains zero UI code. Its only job is to take an **already-open** `mcp.ClientSession` and hand LangGraph a list of tools that look like native LangChain `StructuredTools`.

```
get_mcp_tools(session) → list[BaseTool]

for t in (await session.list_tools()).tools:
    args_schema = json_schema → pydantic
    async def _call(**kwargs):
        await session.call_tool(
            name=t.name, arguments=kwargs)
        yield StructuredTool.from_function(_call)
```

JSON Schema → Pydantic

`_json_schema_to_pydantic` is a tiny converter. It handles the subset of JSON Schema that the tools actually use:

JSON type	Python type
"integer"	<code>int</code>
"number"	<code>float</code>
"boolean"	<code>bool</code>
anything else	<code>str</code>

Required fields become `(type, ...)`; optional ones become `(Optional[type], None)`. This Pydantic model is what LangGraph's LLM sees when it decides to call a tool.

None-filtering

```
clean_kwargs = {k: v for k, v in kwargs.items() if v is not None}
await session.call_tool(name=tool.name, arguments=clean_kwargs)
```

Gemini and other LLMs sometimes include `null` for optional args. The MCP server would receive these as `None` and crash on `arguments.get("x").strip()` style code.

Stripping nulls before the RPC avoids that class of bug.

Transport: streamable HTTP

Both the CLI (`main.py`) and the API (`api_endpoint.py`) use the same pattern:

```
from mcp.client.streamable_http import streamable_http_client
from mcp import ClientSession

async with streamable_http_client(SERVER_URL) as (read, write, _):
    async with ClientSession(read, write) as session:
        await session.initialize()
        langchain_tools = await get_mcp_tools(session)
        # ... run the graph ...
```

`streamable_http_client` opens a long-lived HTTP connection to `http://localhost:8003/mcp` that carries both tool list requests and tool invocations as JSON-RPC frames. The `ClientSession` context manager guarantees the connection is closed when the analysis completes (success or failure).

Why this split?

Concern	Handled in
What tools exist and how they behave	<code>tools/*.py</code>
How to expose them over the network	<code>server.py</code>
How to consume them from LangGraph	<code>client.py</code>
Where in the pipeline each tool is allowed	<code>graph.TOOL_SETS</code>

Each layer has one responsibility and can be swapped independently — e.g. switching from streamable HTTP to stdio only touches `server.py` and the three `streamable_http_client(...)` lines at the call sites.